Структура курса

Конструирование алгоритмов и структур данных

ВЕСЕННИЙ СЕМЕСТР

Продвинутые алгоритмы и структуры данных, базы данных

(17 лек., 34 лаб.)

В рамках курса предусмотрено *17 лекционных занятий*, где подробно разбираются основные концепции и подходы, а также *34 лабораторных занятия*, на которых вы сможете применить полученные знания на практике. Для закрепления материала – *12 лабораторных работ по PostgreSQL*, *6 контестов на алгоритмы* и *14 задач на структуры данных*. В середине семестра проводится *коллоквиум* для проверки теоретических и практических основ первой части курса, а в конце – *экзамен*.

Система оценки построена так, чтобы поощрять систематическую работу и глубокое понимание. В этом курсе нельзя «выехать» на чём-то одном. Успешный разработчик должен одинаково хорошо владеть и теорией, и практикой.

В течение курса оцениваются 5 аспектов вашей работы:

1. Умение работать с базами данных на примере PostgreSQL (лабораторные работы).

2. Умение решать алгоритмические задачи (контесты).

3. Умение проектировать и реализовывать структуры данных (задачи).

4. Понимание теоретических и практических основ первой половины курса (коллоквиум).

5. Понимание теоретических и практических основ курса (экзамен).

Каждый из этих блоков – обязателен. Если вы полностью игнорируете хотя бы один из них, получить положительную оценку за курс будет невозможно. Это не наказание, а отражение реальности: нельзя быть специалистом по алгоритмам, не зная теории или не умея писать код. Экзамен сдают все, потому что это финальная проверка готовности и целостности ваших знаний. Работа в семестре – это не способ «откосить» от экзамена, а способ подготовиться к нему и обеспечить себе отличную итоговую оценку.

Лекции

1. Строки: хеши, префикс-функция, Рабин-Карп, Кнут-Моррис-Пратт, z функция.
2. Автоматы: Ахо-Корасик, бор.
3. Суффиксный массив, суффиксное дерево.
4. Паросочетания: Кун, Эдмондс.
5. Потоки: Форд-Фалкерсон, Эдмондс-Карп.
6. Диниц, масштабирование потока.
7. Задача о назначениях, венгерский алгоритм.
8. Поток минимальной стоимости.
9. ER-модель.
10. Реляционные и дореляционные модели данных.
11. Нормализация отношений.
12. Реляционная алгебра, реляционное исчисление.
13. SQL.
14. Взаимодействие с БД на .NET (CRUD-приложения).
15. Триггеры, резервное копирование.

Лабораторные работы по PostgreSQL

1. Анализ предметной области
2. Даталогическое моделирование БД
3. Создание БД в PostgreSQL
4. Формирование запросов в PostgreSQL
5. Разработка БД для предметной области
6. Разработка запросов к БД
7. Резервное копирование БД
8. Подключение к БД в Visual Studio
9. Построение CRUD приложения
10. Экспорт данных из БД
11. Разработка триггеров
12. Управление правами доступа

Контесты на алгоритмы

7. Графы: DFS, MST.
8. Кратчайшие пути.
9. Алгоритмы на строках.
10. Паросочетания.
11. Потоки.
12. Поток минимальной стоимости.

Задачи на структуры данных

18. Отображение на основе дерева поиска.
19. Множество на основе красно-чёрного дерева.
20. Графы.
21. Отображение на основе хэш-таблицы.
22. Считывание из файла в отображение (HTML-теги).
23. Считывание из файла в отображение (переменные).
24. Сравнение отображения на основе хэш-таблицы и отображения на основе дерева поиска.
25. Множество на основе отображения на основе дерева поиска.
26. Считывание из файла во множество (сравнение строк).
27. Считывание из файла во множество (уникальные слова).
28. Итераторы.
29. Интерфейсы.
30. Строка.
31. Крисс-кросс.

Оценка за лабораторные работы $Оценка_{Лабораторные\ работы}$

Лабораторные работы нацелены на проверку навыков проектирования баз данных, написания SQL-запросов и владения инструментами СУБД PostgreSQL.

Как сдаётся лабораторная работа

Реализация. Студент пишет SQL-скрипты, создающие структуру базы данных и выполняющие манипуляции с данными в соответствии с заданием.

Код-ревью. Код заливается в git-репозиторий для предварительной проверки преподавателем.

Защита у преподавателя:

Оценка 3 – рабочая демонстрация. Скрипты выполняются без синтаксических ошибок. База данных создается, таблицы заполняются, запросы возвращают корректные данные. Студент может объяснить базовую логику написанных запросов.

Оценка 4 – качественное применение. Структура базы данных спроектирована грамотно (корректные типы данных, ограничения целостности). SQL-код хорошо структурирован и отформатирован. Запросы составлены оптимально (например, нет лишних вложенностей, правильно использованы объединения таблиц).

Оценка 5 – глубокое понимание. Студент может объяснить теоретические основы (нормализация, индексы, транзакции), обосновать выбор конкретного решения при проектировании схемы или написании сложного запроса.

Пример защиты лабораторной работы

Преподаватель:

– Здравствуйте! Давайте посмотрим вашу работу по сложным выборкам и объединениям таблиц.

Вопрос 1 (структура). Расскажите про связи между вашими таблицами «Заказы» и «Клиенты». Как реализована связь «один-ко-многим» и зачем здесь нужен внешний ключ?

Вопрос 2 (ключевая концепция). Покажите запрос, где вы используете LEFT JOIN. Чем результат отличался бы, если бы вы написали INNER JOIN, и в каких ситуациях это критично?

Вопрос 3 (теория). Как работает индексирование в PostgreSQL? Если мы часто ищем пользователей по email, какой индекс стоит создать и почему он ускорит поиск?

Вопрос 4 (демонстрация). Напишите прямо сейчас запрос, который выведет топ-3 клиентов по сумме покупок за последний месяц.

Если студент уверенно отвечает, демонстрируя как навыки написания SQL, так и понимание работы СУБД, работа засчитывается.

Если защита не сдана:

– Ваша схема базы данных находится в первой нормальной форме, что приводит к избыточности данных и аномалиям при обновлении. Пожалуйста, нормализуйте схему до третьей нормальной формы и приходите снова.

Важно понимать, что это лишь один из возможных сценариев защиты. ПРЕПОДАВАТЕЛЬ ВПРАВЕ ЗАДАВАТЬ ЛЮБЫЕ ВОПРОСЫ ПО ТЕМЕ, ОТТАЛКИВАЯСЬ ОТ ВАШЕГО КОДА И ОТВЕТОВ, чтобы убедиться в глубине ваших знаний, а не в заучивании конкретных формулировок.

Как формируется $Оценка_{Лабораторные\ работы}$

За каждую из 12 лабораторных работ выставляется отдельная оценка. Итоговая оценка за блок считается как среднее арифметическое этих оценок, с округлением по обычным правилам математики:

$$Оценка_{Лабораторные\ работы} = \frac{\sum_{i=1}^{12} Оценка_{Лабораторная\ работа_i}}{12},$$

где $Оценка_{Лабораторная\ работа_i}$ – оценка за i -ю лабораторную работу.

Оценка за контесты $Оценка_{Контесты}$

Контесты проверяют, умеете ли вы применять алгоритмы на практике. Для получения оценки за контест необходимо решить определённое количество задач и успешно защитить их. Итоговая оценка за каждый контест выставляется по результатам устной защиты.

Как сдаётся контест

1. Решение в системе. Студент решает задачи в системе Codeforces. Решение должно пройти все тесты – это подтверждает корректность и эффективность алгоритма.

2. Оценка за решённые задачи.

В каждом контесте есть M задач, студент решил S из них. Считаем процент выполненных задач:

$$p = \frac{S}{M} \cdot 100\% .$$

Если набрано 90-100% – это оценка 5.

75-89% – оценка 4.

60-74% – оценка 3.

3. Защита у преподавателя.

Дальше, в зависимости от того, на какую оценку вышел по решённым задачам, студент проходит соответствующую защиту:

Оценка 3 – понимание кода. Студент без запинок объясняет любую строчку своего кода, рассказывает, почему выбрал ту или иную структуру данных, объясняет логику решения.

Оценка 4 – теоретическая база. Студент отвечает на 1-2 вопроса по теме контеста. Например, для темы «Сортировки, куча, бинпоиск» вопросы могут быть: «Какова сложность быстрой сортировки в худшем случае и почему?», «В чём принципиальное отличие двоичной кучи от двоичного дерева поиска?».

Оценка 5 – задачи «на подумать». Студент решает 1-2 задачи «на подумать». Например: «На базе кучи постройте структуру данных, которая может находить и удалять медиану ($n/2$-й элемент в отсортированном порядке)», «Предложите алгоритм сортировки циклических сдвигов за $O(n \cdot \log n)$. Указание: зная порядок на отрезках длины L, порядок на отрезках длины $2 \cdot L$ можно восстановить за $O(n)$».

Важно понимать, что это лишь один из возможных сценариев защиты. ПРЕПОДАВАТЕЛЬ ВПРАВЕ ЗАДАВАТЬ ЛЮБЫЕ ВОПРОСЫ ПО ТЕМЕ, ОТТАЛКИВАЯСЬ ОТ ВАШЕГО КОДА И ОТВЕТОВ, чтобы убедиться в глубине ваших знаний, а не в заучивании конкретных формулировок.

Пример защиты контеста

Преподаватель:

– Здравствуйте! Вижу, вы решили 6 из 8 задач в контесте по динамическому программированию. Отлично. Давайте обсудим задачу «Наибольшая общая подпоследовательность».

Вопрос 1 (суть алгоритма). Расскажите, как вы решали эту задачу. Какая идея лежит в основе решения? Что хранится в вашей DP-таблице dp[i][j]?

Вопрос 2 (сложность). Оцените асимптотику вашего решения по времени и памяти. Почему именно такая?

Вопрос 3 (код). Откройте ваш код. Объясните, что происходит в строках с 25 по 32. Зачем тут двойной цикл?

Вопрос 4 (связь с теорией). На какой лекции разбиралась эта тема? Какие ещё классические задачи на ДП обсуждались на лекции 6?

Вопрос 5 («на подумать»). Ваш алгоритм ищет длину НОП. Как его изменить, чтобы восстановить саму подпоследовательность? А если бы нужно было найти не наибольшую, а наименьшую общую подпоследовательность – имеет ли такая задача смысл?

Если студент уверенно отвечает на все вопросы – задача засчитана. Преподаватель может проверить 2-3 задачи из контеста. Если студент «плавает», не может объяснить код или идею – контест не засчитывается.

Если контест не сдан:

– Вам нужно лучше разобраться в теме. Повторите лекцию, проанализируйте свой код. Можете прийти на пересдачу на следующей неделе. Без успешной защиты решённые задачи не учитываются.

Важно понимать, что это лишь один из возможных сценариев защиты. ПРЕПОДАВАТЕЛЬ ВПРАВЕ ЗАДАВАТЬ ЛЮБЫЕ ВОПРОСЫ ПО ТЕМЕ, ОТТАЛКИВАЯСЬ ОТ ВАШЕГО КОДА И ОТВЕТОВ, чтобы убедиться в глубине ваших знаний, а не в заучивании конкретных формулировок.

Как формируется $Оценка_{Контесты}$

За каждый из 6 контестов выставляется отдельная оценка по итогам его защиты. Итоговая оценка за блок контестов $Оценка_{Контесты}$ считается как среднее арифметическое этих оценок, с округлением по обычным правилам математики:

$$Оценка_{Контесты} = \frac{\sum_{i=1}^{6} Оценка_{Контест_i}}{6},$$

где $Оценка_{Контест_i}$ – оценка за i-й контест.

Оценка за задачи $Оценка_{Задачи}$

Здесь основной упор делается на качество кода и понимание принципов ООП.

Как сдаётся задача

1. Реализация. Студент пишет код, реализующий структуру данных по заданию.

2. Код-ревью. Код заливается на git-репозиторий. Преподаватель может посмотреть и оставить комментарии.

3. Защита у преподавателя:

Оценка 3 – рабочая реализация. Студент показывает код, который компилируется без ошибок и реализует все методы из задания. Преподаватель проверяет работу на примерах.

Оценка 4 – качество кода. Код структурирован: классы, методы и поля логично организованы. Есть комментарии к сложным местам. Соблюдается единый стиль именования (camelCase для методов, PascalCase для классов).

Оценка 5 – объяснение реализации. Студент может объяснить ключевые методы структуры данных и их временные/пространственные сложности.

Важно понимать, что это лишь один из возможных сценариев защиты. ПРЕПОДАВАТЕЛЬ ВПРАВЕ ЗАДАВАТЬ ЛЮБЫЕ ВОПРОСЫ ПО ТЕМЕ, ОТТАЛКИВАЯСЬ ОТ ВАШЕГО КОДА И ОТВЕТОВ, чтобы убедиться в глубине ваших знаний, а не в заучивании конкретных формулировок.

Пример защиты задачи

Преподаватель:

– Добрый день! Давайте посмотрим вашу реализацию двусвязного списка. Откройте, пожалуйста, код.

Вопрос 1 (структура). Расскажите, как устроен ваш класс. Какие приватные поля вы используете и зачем они нужны? Как устроен узел списка?

Вопрос 2 (ключевые методы). Давайте разберём метод add. Прокомментируйте его по шагам. Какие граничные случаи вы предусмотрели (добавление в начало, в конец, в пустой список)?

Вопрос 3 (сложность). Какая сложность у addLast? А у addFirst? Почему так?

Вопрос 4 (сравнение). Вы делали похожую задачу на динамический массив MyArrayList. В чём плюсы и минусы списка по сравнению с массивом? Приведите пример, где список будет лучше, а где – хуже.

Вопрос 5 (демонстрация). Напишите прямо сейчас небольшой main: создайте список MyLinkedList, добавьте 5 элементов, удалите третий, выведите содержимое на экран.

Если студент уверенно отвечает на вопросы, понимает инварианты структуры и умеет сравнивать с другими решениями – задача засчитана.

Если защита не сдана:

– Ваш remove неправильно работает при удалении последнего элемента. Исправьте и приходите ещё раз. Без этого задачу не зачту.

Как формируется $Оценка_{Задачи}$

За каждую практическую задачу выставляется отдельная оценка, отражающая качество её реализации и защиты. Итоговая оценка за блок задач $Оценка_{Задачи}$ считается как среднее арифметическое этих оценок, с округлением по обычным правилам математики:

$$Оценка_{Задачи} = \frac{\sum_{i=1}^{14} Оценка_{Задача_i}}{14},$$

где $Оценка_{Задача_i}$ – оценка за i-ю задачу.

Оценка за коллоквиум $Оценка_{Коллоквиум}$

Коллоквиум проводится для того, чтобы проверить, как вы усвоили материал первой половины курса. Это помогает выявить, кто отстаёт, а также мотивирует всех повторить и систематизировать пройденное. Кроме того, это тренировка перед экзаменом.

Как оценивается работа в первой половине семестра

За работу в первой половине семестра студенты могут получить до 50 баллов. Эти баллы складываются из выполнения лабораторных работ по ООП, решения задач на структуры данных и участия в контестах по алгоритмам. Для получения максимального количества баллов необходимо сдать 2 лабораторные работы по ООП, выполнить 4 задачи на структуры данных и принять участие в 2 контестах по алгоритмам. Каждая лабораторная работа оценивается в 5 баллов, каждая задача на структуры данных – в 5 баллов, каждый контест – в 10 баллов.

Как проходит коллоквиум

В билете 2 теоретических вопроса и 3 практических задачи. Каждый теоретический вопрос оценивается в 10 баллов, каждая практическая задача – также в 10 баллов. Теоретические вопросы охватывают материал лекций первой половины семестра. Практические задачи включают задачу на алгоритмы, задачу на структуры данных и задачу на объектно-ориентированное программирование. Для получения максимальной оценки студент должен продемонстрировать глубокое понимание теории и умение применять знания на практике.

Как формируется $Оценка_{Коллоквиум}$

За каждый вопрос и задачу ставится отдельная оценка. Итоговая оценка за коллоквиум $Оценка_{Коллоквиум}$ считается как среднее арифметическое этих оценок, с округлением по обычным правилам математики:

$$Оценка_{Коллоквиум} = \frac{Оценка_{Вопрос_1} + Оценка_{Вопрос_2} + \sum_{i=1}^{3} Оценка_{Задача_i}}{5},$$

где $Оценка_{Вопрос_1}$ – оценка за первый теоретический вопрос, $Оценка_{Вопрос_2}$ – оценка за второй теоретический вопрос, $Оценка_{Задача_i}$ – оценка за i-ю задачу.

Оценка за экзамен $Оценка_{Экзамен}$

Экзамен – это итоговая проверка знаний, которая охватывает весь материал, пройденный за семестр. Его цель – оценить, насколько глубоко и системно вы усвоили курс, и проверить ваше умение связывать разные темы в единое целое.

Как проходит экзамен

Формат похож на коллоквиум, но вопросы уже по всему курсу.

В билете 2 теоретических вопроса (один – по первой половине курса, второй – по второй) и 3 практических задачи.

Преподаватель может дополнительно задавать вопросы по всему курсу.

Как формируется $Оценка_{Экзамен}$

За каждый вопрос и задачу ставится отдельная оценка. Итоговая оценка за экзамен $Оценка_{Экзамен}$ считается как среднее арифметическое этих оценок, с округлением по обычным правилам математики:

$$Оценка_{Экзамен} = \frac{Оценка_{Вопрос_1} + Оценка_{Вопрос_2} + \sum_{i=1}^{3} Оценка_{Задача_i}}{5},$$

где $Оценка_{Вопрос_1}$ – оценка за первый теоретический вопрос, $Оценка_{Вопрос_2}$ – оценка за второй теоретический вопрос, $Оценка_{Задача_i}$ – оценка за i-ю задачу.

Итоговая оценка *Итог*

Компоненты оценки:

1. $Оценка_{Лабораторные\ работы}$ – за все 5 лабораторных работ.
2. $Оценка_{Контесты}$ – за все 8 контестов на алгоритмы.
3. $Оценка_{Задачи}$ – за все 17 задач на структуры данных.
4. $Оценка_{Коллоквиум}$ – за коллоквиум.
5. $Оценка_{Экзамен}$ – за экзамен.

Как формируется *Итог*

1. Стандартный случай (все блоки сданы).

Если у студента есть ненулевые оценки по всем пяти компонентам, итог считается как среднее арифметическое:

$$Итог = (Оценка_{Лабораторные\ работы} + Оценка_{Контесты} + Оценка_{Задачи} + \\ + Оценка_{Коллоквиум} + Оценка_{Экзамен}) / 5.$$

2. Штрафной случай (один из блоков провален).

Если хотя бы по одному из пяти ключевых компонентов (лабораторные работы, контесты, задачи, коллоквиум или экзамен) стоит 0, это значит, что минимальные требования по этому направлению не выполнены.

Что такое 0?

0 ставится, если сдано меньше 60% работ в блоке (то есть меньше 3 лабораторных работ, меньше 5 контестов или меньше 11 задач). Это не оценка 2, а именно индикатор того, что блок не выполнен.

В таком случае итог считается так – добавляется ещё один фиктивный 0:

$$Итог = (Оценка_{Лабораторные\ работы} + Оценка_{Контесты} + Оценка_{Задачи} + \\ + Оценка_{Коллоквиум} + Оценка_{Экзамен} + 0) / 6.$$

Оценка 2 – $Итог < 2,5$

Оценка 3 – $2,5 \le Итог < 3,5$

Оценка 4 – $3,5 \le Итог < 4,5$

Оценка 5 – $Итог \ge 4,5$

Пример

Студент отлично сдал лабораторные работы, задачи, коллоквиум, экзамен, но полностью проигнорировал контесты.

Считаем:

$$Итог = \frac{5+0+5+5+5+0}{6} = \frac{20}{6} = 3,(3) \approx 3.$$

Максимум, на что он может рассчитывать – это оценка 3, и то, если всё остальное сдано на 5.

ЛЮБОЕ СПИСЫВАНИЕ И ПЛАГИАТ, включая копирование кода из интернета, у одногруппников или с помощью генераторов вроде ChatGPT, DeepSeek и других ИИ-инструментов, СТРОГО ЗАПРЕЩЕНЫ. Если такое обнаружится, работа АННУЛИРУЕТСЯ, а студенту могут быть назначены ДИСЦИПЛИНАРНЫЕ МЕРЫ. Обсуждать идеи между собой РАЗРЕШАЕТСЯ, но финальные ответы и код должны быть ВАШИМИ СОБСТВЕННЫМИ.

Учёт посещаемости и конспектов

Посещение занятий и ведение конспектов – важная часть учебного процесса. Хотя основное внимание уделяется вашим результатам, систематическое пренебрежение занятиями не останется без последствий.

Конспекты

Допускаются два формата ведения конспекта: традиционный рукописный в тетради или цифровой, созданный от руки с помощью стилуса на планшете.

Наличие конспекта является обязательным условием допуска к коллоквиуму и экзамену.

Штрафы за пропуски

Посещаемость лекций и лабораторных занятий влияет на итоговую оценку.

За каждый пропуск лекции или лабораторного занятия без уважительной причины (без официального документа) из итогового расчётного показателя $Итог$ вычитается 0,05.

Суммарный штраф за все пропуски в течение семестра не может превышать 1.

Пример

Студент получил расчётный показатель $Итог = 4,6$, но пропустил 4 лекции и 2 лабораторных занятия без уважительной причины (всего 6 пропусков). Его итоговый показатель будет скорректирован:

$$Итог = 4,6 - 6 \cdot 0,05 = 4,6 - 0,3 = 4,3.$$

Итог – оценка 4.

Система бонусов за академические достижения

Мы поощряем студентов, которые применяют знания на практике и выходят за рамки обязательной программы курса. Бонусы позволяют улучшить ваши оценки за работу в семестре.

ContestSFedU и IT-Планета

Студент, прошедший на финальный тур турнира ContestSFedU и занявший призовое место или прошедший в финал конкурса СУБД Postgres Pro Международной олимпиады «IT-Планета 2026», автоматически получает оценку 5 за курс. Эти достижения подтверждают исключительный уровень владения материалом и полностью покрывают цели обучения.

Повышение оценок за достижения

Научные публикации, доклады и победы или призовые места на олимпиадах позволяют повысить оценку за одну из сданных работ (лабораторную работу, контест или задачу на структуры данных).

Научная публикация по теме курса позволяет повысить оценку за одну работу на 2 пункта (например, с 3 до 5).

Выступление с докладом на научной конференции позволяет повысить оценку за одну работу на 1 пункт (например, с 4 до 5).

Победа или призовое место на соревновании по программированию (регионального уровня) позволяют получить оценку 5 за один любой контест.